E-commerce Order Fulfillment Automation Backend
Final Project Report

Course
SWE 4304 - Software Project Lab-I

Submission Date
30 March 2026

Supervisor
Anika Farzana
Junior Lecturer, Department of Computer Science and Engineering

Team / Group
Group 13

Members
Saika Sarara (230042159)
Adiba Ahsan Raiyan (230042161)
Maliha Tasnim Khan (230042127)

Project Type
Business-side command-line backend for e-commerce order fulfillment

Table of Contents

1. Executive Summary
2. Problem Statement
3. Potential Solution and Design Choices
4. Implemented Features
5. Tools and Technology
6. Application Domain
7. System Architecture and Class Overview
8. Project Timeline
9. Contribution
10. Future Work
11. Source Repository
12. Conclusion

1. Executive Summary

The E-commerce Order Fulfillment Automation Backend is a Java-based command-line application developed to automate the internal order-processing workflow of a small to medium e-commerce business. Instead of focusing on customer-facing browsing or checkout interfaces, the system concentrates on the backstage workflow that begins after an order is placed. The application validates orders, checks product availability, simulates payment processing, generates invoices and receipts, updates shipping stages, writes workflow logs, and generates operational reports.

The project was designed as a headless application in accordance with the course requirement of building a non-GUI software system. It uses modular object-oriented design so that major responsibilities are separated across dedicated classes such as Admin, Workflow, DataPersistence, PaymentService, Log, Order, Item, Product, and Role. This structure makes the project easier to test, explain, and extend.

In its current form, the system provides a role-aware admin dashboard, JSON-based data persistence, order and stock management features, audit logging, report generation, simulation scenarios, reorder and retry support, stale-order auto-cancellation, and administrative operations such as archiving delivered orders, restoring history, clearing logs, and changing admin credentials. The final result is a practical academic backend that demonstrates workflow automation, team-based implementation, and structured code organization.

2. Problem Statement

Many small e-commerce businesses still handle fulfillment manually after a customer places an order. This creates repeated operational problems: order details may be entered incorrectly, stock can be mismatched, invoices may be skipped, shipment updates may be delayed, and there is often no clean audit trail to explain what happened to a specific order. During high-volume periods such as festival seasons, these weaknesses become more visible because the business must process many orders quickly with limited staff.

Our project addresses the observation that the real bottleneck in many online shops is not the storefront itself, but the internal workflow that connects validation, stock checking, payment, packaging, shipment progression, and reporting. Without automation, businesses risk overselling unavailable products, approving invalid requests, cancelling orders without proper reasons, and losing visibility into

operational performance. The problem, therefore, is the lack of a structured, traceable, and reliable backend fulfillment process.

3. Potential Solution and Design Choices

To solve this problem, we developed a business-side backend system that automates the internal fulfillment pipeline through a menu-driven CLI environment. The design choice to use a command-line application was intentional: it satisfies the course requirement for a headless application and keeps the implementation focused on workflow logic rather than interface decoration. Administrators interact with the system through a controlled dashboard, while the backend manages validation, inventory verification, stock reservation, payment simulation, invoice generation, shipment status progression, logging, and reporting.

The project follows an object-oriented design approach. Data entities such as orders, products, items, roles, and admins are represented through separate classes. Workflow orchestration is handled centrally through Workflow.java, while file loading and saving are managed by DataPersistence.java. Payment behavior is isolated in PaymentService.java, and order-event recording is handled by Log.java. This separation improves maintainability and helps each team member work on a well-defined part of the project.

Another major design choice was to use local JSON files for persistence. This makes the system easier to demonstrate, portable across environments, and suitable for an academic setting where the goal is to show logic and structure clearly without requiring external database setup. Security was handled by hashing passwords with SHA-256, and role support was added to distinguish between ADMIN, MANAGER, and SUPPORT access levels.

4. Implemented Features

4.1 Core Workflow Features

- Secure Admin Login with SHA-256 hashed password verification, masked password input, and login-attempt auditing.
- Role-aware admin dashboard supporting ADMIN, MANAGER, and SUPPORT access levels with permission-based menu restriction.
- Order Intake through manual order creation, reorder and retry flows, JSON-based bulk import, and test-data loading support.
- Order Validation to reject incomplete orders, invalid quantities, duplicate same-order selections, unsupported payment modes, or malformed import data.
- Inventory Verification and Reservation before order acceptance, with stock rollback when payment fails.
- Payment Simulation supporting COD and MockCard approval/decline flow.
- Invoice generation during successful order processing.
- Shipping Progression through PENDING, PACKED, SHIPPED, OUT_FOR_DELIVERY, and DELIVERED statuses, with tracking ID creation at shipment stage.
- Workflow Logging, order-specific log viewing, and timeline display through logs.json.
- Summary Reporting, system health checking, and automatic cancellation of stale pending orders with recent auto-cancel review.

4.2 Product and Operational Features

- Advanced order search and filter by order ID, status, payment mode, and date.
- Advanced product filtering by brand and category.
- Product management: add, edit, delete, and restock products.
- Low-stock alert display and stock report export.
- Receipt generation with console preview for delivered orders.
- Simulation mode for successful, payment-failure, inventory-shortage, and random scenarios.
- Bulk import of orders from orders_import.json.
- Archive delivered orders older than a selected age threshold.
- Delete all order history with backup creation, restore archived orders by latest session, date, or full archive, and undo the last restore.
- Administrative operations such as adding a new admin, changing passwords securely, and clearing logs.

Together, these features show that the project covers both the main fulfillment pipeline and the surrounding administrative controls that a real business backend typically requires.

5. Tools and Technology

The project was implemented in Java and runs as a console-based Command Line Interface (CLI) application. The overall architecture is modular, object-oriented, and workflow-driven. Git and GitHub were used for source control, while local JSON files were used to store persistent data such as products, orders, admins, logs, login audits, invoices, reports, stock reports, archive backups, and receipts.

A special implementation constraint of the project was to keep library dependence very limited. According to the current implementation approach, five core Java methods were intentionally emphasized throughout the codebase: System.out.print() for console output, BufferedReader.readLine() for user input and file reading, FileWriter.write() for file writing, String.split() for structured text parsing, and MessageDigest.digest() for password hashing.

This limited-method approach required several features to be built manually, including JSON string parsing, array splitting, serialization, deserialization, ID normalization, and workflow validation logic. That constraint increased the learning value of the project because it forced the team to think carefully about how common backend tasks actually work under the hood.

6. Application Domain

This project belongs to the e-commerce operations and backend automation domain. More specifically, it targets order fulfillment management: the internal sequence of checks and updates that happens after a customer submits an order. The application is not a customer portal; instead, it supports administrative staff who need to validate orders, verify stock, process payment status, track shipment stages, keep logs, and produce summary reports.

Because the application is headless and workflow-centered, it is especially suitable as an educational example of how backend business logic can be designed independently from a graphical user interface. The same design could later be extended into a web dashboard, service layer, or API-backed business system.

7. System Architecture and Class Overview

The system is organized around a small set of focused classes. The following table summarizes the responsibility of each implemented class in the current version of the project.

Class	Primary Responsibility
Main.java	Entry point of the application; initializes data loading, logging, admin login, dashboard loop, and final saving.
Workflow.java	Main controller of the backend; handles dashboard options, order processing, search and filter, reports, receipts, reorder/retry flow, auto-cancellation, archives, simulation, and administrative actions.
DataPersistence.java	Loads and saves JSON data, generates and normalizes IDs, computes totals, stores login audit information, supports test-data loading, and manages file paths and restore backups.
PaymentService.java	Simulates payment logic for COD and MockCard modes and writes payment outcomes to the log.
Admin.java	Represents admin users, authenticates login, manages roles, masks password input, and hashes passwords using SHA-256.
Log.java	Appends workflow events to logs.json and displays order-specific timelines.
Order.java	Stores order data such as items, address, payment mode, status, total amount, cancellation reason, tracking ID, and simulation-related metadata.
Item.java	Stores product ID and quantity for each order item.
Product.java	Stores product catalog information such as product ID, category, brand, name, price, and stock.
ManualInputReader.java	Custom console input reader that reads characters from System.in and returns input line by line for the CLI workflow.
Role.java	Defines permission levels: ADMIN, MANAGER, and SUPPORT.

The persistence layer currently works with JSON files such as admins.json, products.json, orders.json, logs.json, login_audit.json, invoices.json, stock_report.json, report.json, archive_orders.json, orders_restore_backup.json, and receipt files generated during operation. This design keeps the project easy to run and demonstrate while still preserving data across program executions.

8. Project Timeline

The project followed the semester plan proposed earlier in the course. The timeline below summarizes the progression from problem identification to final refinement.

Week	Planned Activity
Week 1	Problem identification and requirement analysis
Week 2	Feature planning and project scoping

Week 3	Flowchart and UML class diagram design
Week 4	Proposal submission and presentation
Week 5	Core entity class implementation
Week 6	Order validation and inventory logic
Week 7	Stock reservation and cancellation handling
Week 8	Invoice generation and payment simulation
Week 9	Shipping workflow and tracking logic
Week 10	Admin dashboard and order management
Week 11	Data persistence and serialization/deserialization
Week 12	Security features: password hashing and admin login
Week 13	Logging system and summary report generation
Week 14	Testing, debugging, and documentation
Week 15	Progress presentation
Week 16	Final refinement and final presentation

The actual implementation evolved with feedback by clarifying the project as backend-only, removing customer-facing scope, strengthening authentication, and adding more reporting and operational controls.

9. Contribution

The implementation work was divided by class ownership while planning, debugging, testing, documentation, and final preparation were handled collectively by all team members. The class-wise contribution is summarized below.

Member	Implemented Classes	Main Contribution
Saika Sarara 230042159	Workflow.java Main.java ManualInputReader.java	Led workflow orchestration, admin dashboard behavior, menu handling, order search/filter, receipt/report/archive operations, simulation-related flow, stale-order handling, custom console input support, and the overall application startup/shutdown logic.
Maliha Tasnim Khan 230042127	DataPersistence.java PaymentService.java	Implemented JSON-based loading and saving, ID normalization and generation, total calculation, login audit writing, backup/test-data support, and payment simulation for COD and MockCard.
Adiba Ahsan Raiyan 230042161	Admin.java Item.java Log.java Order.java Product.java	Implemented authentication and password hashing, role-aware admin model, core entity classes, workflow log writing, password masking support, and order timeline

	Role.java	viewing support.

Beyond class ownership, the group collaborated in requirement discussion, scope refinement, debugging, testing, README/report writing, and preparation for proposal, progress, and final submission.

10. Future Work

- Add a graphical or web-based frontend for easier interaction and broader usability.
- Replace local JSON persistence with a relational database for better scalability and concurrent access control.
- Add customer management, order history analytics, and richer invoice/receipt generation.
- Integrate real payment gateways and courier APIs instead of simulation-only processing.
- Strengthen role-based authorization and add more granular permission control.
- Expand automated testing and validation to improve robustness under larger workloads.

These improvements would transform the current academic prototype into a more production-like operations platform while preserving the same workflow-centered architecture.

11. Source Repository

GitHub Repository Link: https://github.com/saikasarara645-bot/E-commerce-Order-Fulfillment-Automation-Backend

12. Conclusion

The E-commerce Order Fulfillment Automation Backend demonstrates how a real business-side order pipeline can be modeled with clean workflow logic, modular code organization, and persistent storage in a headless Java environment. By concentrating on validation, stock verification, payment simulation, invoice and receipt support, status progression, logging, reporting, and admin control, the project solves a practical operational problem rather than only presenting a user-facing interface.

As a course project, it also reflects important software engineering outcomes: problem analysis, design choice justification, teamwork, code organization, and documentation. The project therefore serves both as a working backend prototype and as evidence of the team’s ability to build a structured multi-class software system from proposal to final delivery.